PROJECT REPORT

FOR

Foundations of Artificial Intelligence (CS23533)

AI DISASTER RELIEF RESOURCE OPTIMIZER

A PROJECT REPORT

Submitted by

2116240701456 – SAICHUPAK E

2116240701444 – RUPAKANTHAN A



November, 2026

BONAFIDE CERTIFICATE

Certified that this project report "AI DISASTER RELIEF RESOURCE OPTIMIZER" is the bonafide work of "SAICHUPAK E (240701456) & RUPAKANTHAN A (240701444)" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**4**
1	**INTRODUCTION**	**5**
2	**LITERATURE REVIEW**	**7**
3	**PROPOSED SYSTEM**	**10**
	3.1 ARCHITECTURE DIAGRAM	11
	3.2 TECHNOLOGIES USED	12
	3.3 ALGORITHMS USED	13
4	**MODULES DESCRIPTION**	**14**
	4.1 User Authentication Module	14
	4.2 Live Disaster Data Module	14
	4.3 Disaster Management Module	14
	4.4 Hospital Module	15
	4.5 Shelter Module	15
	4.6 Warehouse Module	16
	4.7 Rescue Team Dispatch Module	16
	4.8 Ticketing and Resource Planning Module	16
	4.9 User Interface Module	17
5	**IMPLEMENTATION AND RESULTS**	**18**
	5.1 EXPERIMENTAL SETUP	18
	5.2 RESULTS	19
6	**CONCLUSION AND FUTURE WORK**	**23**
7	**REFERENCES**	**25**

ABSTRACT

Natural disasters such as floods, cyclones, earthquakes, wildfires and landslides often strike several places at the same time. In the first hours after such an event, relief workers must decide where the danger is greatest, which hospital can receive the injured fastest, where families can find safe shelter, and from which warehouse food and water should be sent. When this information is scattered across different websites and registers, decisions become slow and supplies may not reach the people who need them first.

This project, AI Disaster Relief Resource Optimizer (Reliefline Live), brings these decisions together in one web-based decision-support system built on a live world map. It reads real disaster reports from GDACS, USGS and NASA EONET and classifies each event as High, Medium or Low risk. The user can open five categories: Disaster Management, Hospitals, Shelters, Warehouses and Rescue Teams. When an emergency is marked on the map, the system finds the nearest hospital by road travel time and draws the fastest route in blue. Food, water, medical and shelter needs are estimated for each affected area and tracked as tickets. A genetic algorithm allocates warehouse stock to the highest-risk areas first, and the Hungarian algorithm assigns rescue teams to simultaneous incidents.

The frontend uses HTML, CSS, JavaScript and Leaflet, and the backend uses Python and FastAPI. Locations come from OpenStreetMap and routes from OSRM, so no paid map service is needed.

CHAPTER 1
INTRODUCTION

Countries such as India are exposed to many natural hazards, including floods, cyclones, earthquakes and landslides. When one of these events occurs, the first hours decide how many lives can be saved. Rescue and relief teams must quickly find out where the damage is, how many people are affected, which hospitals can treat the injured, where families can stay safely, and how much food, water and medicine must be delivered. These decisions are usually taken under pressure and with incomplete information.

In practice, the information needed is spread across many places. Hazard alerts are published on one website, hospital and shelter locations are kept in separate lists, warehouse stock is recorded in registers, and the progress of each relief request is tracked by phone calls or paper notes. As a result, the dispatcher must combine all of these sources by hand. This is slow, it can lead to mistakes, and it often means that the area with the greatest need is not served first.

The proposed system, AI Disaster Relief Resource Optimizer, also called Reliefline Live, solves this problem by bringing the main relief decisions into one web application that is built on a live world map. The system collects real disaster events from open data services and shows them as coloured markers according to their risk level. From a single search box the user can open five categories: Disaster Management, Hospitals, Shelters, Warehouses and Rescue Teams. Each category shows the relevant places on the map and opens a detail page when a marker is clicked.

The emergency feature is the central part of the project. When a user clicks a point on the map, the system treats it as an emergency, finds the nearest hospitals, compares their real road travel times, and draws the fastest route as a blue line, in the same way as a navigation application. In the same way, the shelter module

suggests safe shelters outside the danger zone, and the warehouse module shows the stores from which supplies can be sent.

The project applies several techniques from the Foundations of Artificial Intelligence course. Search algorithms are used to find the shortest route to a hospital. A genetic algorithm, which is a local search and optimisation method, allocates limited warehouse stock to the areas with the highest risk. The Hungarian algorithm assigns rescue teams to simultaneous incidents so that the total travel distance is minimum. A Random Forest classifier predicts the risk level of an area, and a rule-based method classifies each event as High, Medium or Low. All of this is delivered through a simple interface so that an officer without technical knowledge can use it.

The objectives of the project are:

- To collect real, current disaster events from GDACS, USGS and NASA EONET and show them on an interactive world map.
- To classify every event as High, Medium or Low risk and show its impact area.
- To find the nearest hospital by road travel time for an emergency marked on the map and draw the fastest route.
- To identify safe shelters outside the danger zone and show the evacuation route to them.
- To show warehouses and estimate the food, water, medical and shelter supplies needed for each affected area.
- To allocate limited warehouse stock to the highest-risk areas using a genetic algorithm.
- To assign rescue teams to simultaneous incidents using the Hungarian algorithm.To track every relief request as a ticket and present all details in a user-friendly web interface.

CHAPTER 2
LITERATURE REVIEW

Disaster operations management has been studied for many years. Altay and Green (2006) reviewed operations research and management science work on disasters and organised it by the phases of mitigation, preparedness, response and recovery. Galindo and Batta (2013) updated this survey and showed that most research is still concerned with mathematical models, while few studies describe tools that a relief officer can actually use in the field. This is the first gap addressed by the present project: a model is useful only when it is delivered through a clear and usable interface.

Several authors have studied the movement of relief goods. Özdamar, Ekinci and Küçükyazici (2004) proposed a multi-period model for emergency logistics, which plans the dispatch of vehicles to deliver relief items. Sheu (2007) described a method for quick-response distribution of relief to areas with urgent demand. These works show that the allocation of limited stock is a central problem. However, they are presented as optimisation models and do not provide a live map, a ticket workflow, or connected information about hospitals and shelters.

Reliable and current hazard information is the starting point of any response. The Global Disaster Alert and Coordination System, GDACS (n.d.), provides alerts for earthquakes, tropical cyclones, floods, volcanoes and forest fires, and rates them as Green, Orange or Red. The United States Geological Survey, USGS (n.d.), publishes machine-readable earthquake feeds that are updated within minutes. The Earth Observatory Natural Event Tracker of NASA (n.d.) lists open events such as wildfires, severe storms and floods. Each source covers only a part of the picture, so a practical system must combine them into one common format, which is what the proposed system does.

Open geographic data has made it possible to build such systems without paid services. Haklay and Weber (2008) described OpenStreetMap, a map built by

volunteers that includes roads, hospitals, schools and community buildings. The Overpass API (n.d.) allows these features to be read for any area. Luxen and Vetter (2011) presented the Open Source Routing Machine, OSRM, which calculates driving routes and travel times on OpenStreetMap roads in real time. A limitation of volunteer data is that it may be incomplete or out of date, so the results should be treated as decision support and not as official records.

Finding the nearest facility is a shortest-path problem. Dijkstra (1959) gave the classical algorithm for the shortest path in a weighted graph. Hart, Nilsson and Raphael (1968) introduced the A* algorithm, which adds an admissible heuristic, such as the straight-line distance to the goal, so that fewer nodes are expanded. Russell and Norvig (2021) discuss these uninformed and informed search strategies as the basis of problem-solving agents. The proposed system uses road-network shortest paths for hospital routing and an A* implementation for route planning in its backend.

Allocation of limited resources is an optimisation problem. Holland (1975) introduced genetic algorithms, which improve a population of candidate solutions through selection, crossover and mutation. Such local search methods are suitable when the number of possible allocations is very large and an exact method is not practical. For assigning teams to incidents, Kuhn (1955) proposed the Hungarian method and Munkres (1957) described how it can be implemented to solve the assignment problem in polynomial time. In this project the genetic algorithm distributes warehouse stock to zones in order of risk, and the Hungarian algorithm assigns rescue teams so that the total distance is minimum.

Machine learning can support risk assessment. Breiman (2001) described random forests, which combine many decision trees and give accurate classification with little tuning. Pedregosa et al. (2011) provide an implementation of this method in the scikit-learn library for Python. The proposed system includes a Random

Forest classifier that predicts the High, Medium or Low risk of an area from factors such as rainfall, water level, elevation and population density.

Finally, relief quantities must follow accepted standards. The Sphere Association (2018) sets minimum standards for humanitarian response, including a minimum quantity of drinking and domestic water for each person every day. The proposed system uses such figures to estimate the water, food and other supplies needed for the number of people affected.

From the reviewed literature it can be seen that hazard data, open maps, routing engines, optimisation algorithms and humanitarian standards all exist, but they are rarely combined in one tool. The proposed AI Disaster Relief Resource Optimizer fills this gap by bringing live hazard data, hospital routing, shelters, warehouses, rescue team assignment and ticket tracking into a single, low-cost web application that a relief officer can operate directly.

CHAPTER 3
PROPOSED SYSTEM

The proposed system, Reliefline Live, is a client–server web application. The client is a web page written in HTML, CSS and JavaScript that displays a Leaflet map. The server is a Python application built with FastAPI that runs the optimisation algorithms and manages user accounts. The user chooses a category from the search box, clicks on the map, and the system returns the markers, routes, supply lists and tickets that are needed. The main features of the system are:

- Live data: real disaster events are read from GDACS, USGS and NASA EONET every time the page is opened.
- Risk levels: every event is shown as High, Medium or Low with its impact area.
- Click-to-mark emergency: a click on the map marks an emergency and shows the nearest hospital, ranked by road time.
- Five categories: Disaster Management, Hospitals, Shelters, Warehouses and Rescue Teams in one search box.
- Supply planning: food, water, medical and shelter requirements are calculated for each affected area.
- Ticketing: damage and resource requests are tracked as tickets with priority, team and status.
- Low cost: only free and open services are used, so no paid map or AI service is needed.

3.1 ARCHITECTURE DIAGRAM

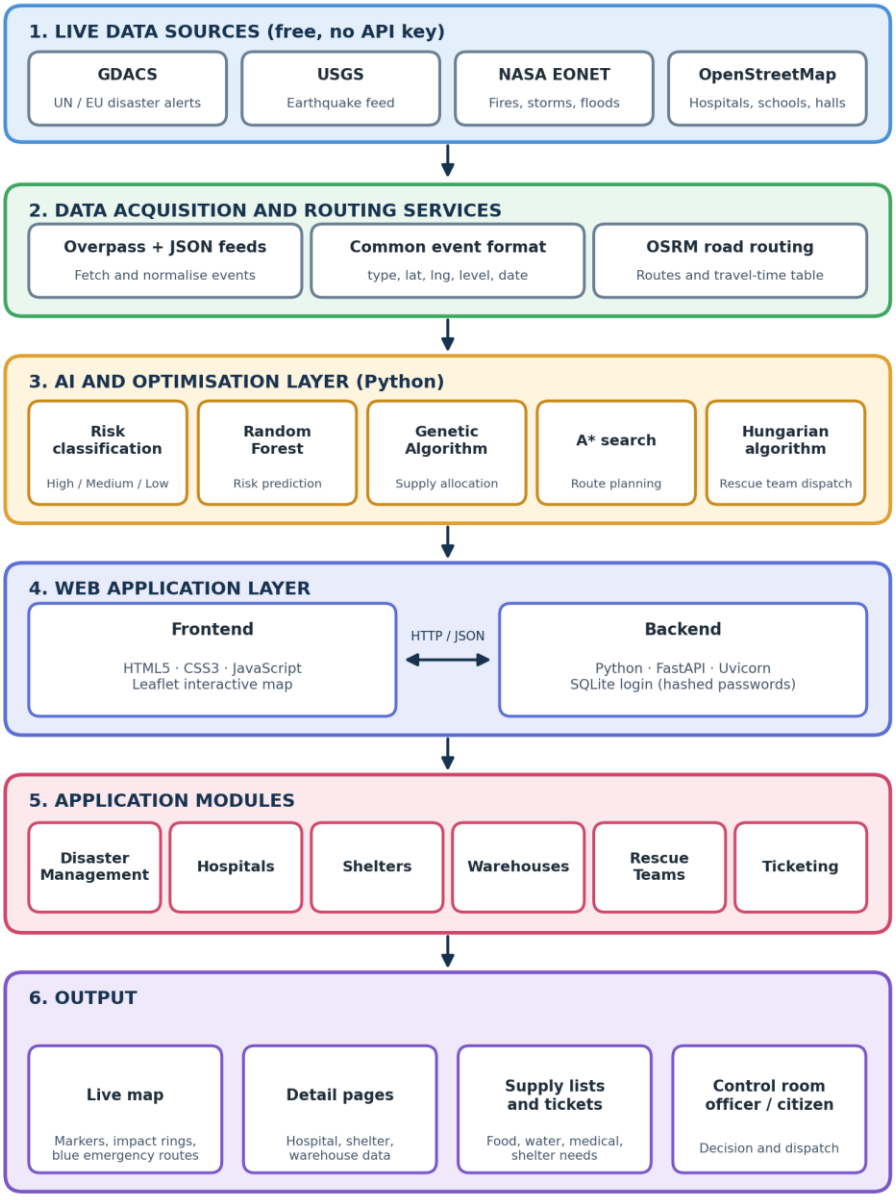


Figure 3.1 Architecture of the AI Disaster Relief Resource Optimizer

The architecture has six layers. Live data sources supply disaster events and map features. The acquisition layer reads them in JSON form and converts them to a common event format with type, location, risk level and date, and also calls the OSRM service for routes and travel times. The AI layer, written in Python, contains the risk classification, Random Forest, genetic algorithm, A* search and Hungarian algorithm. The application layer connects the Leaflet frontend with

the FastAPI backend over HTTP. The modules layer groups the features, and the output layer presents maps, detail pages, supply lists and tickets to the officer.

3.2 TECHNOLOGIES USED

Table 3.1 Technologies used

S.No.	Technology	Purpose
1	HTML5	Creates the structure of the web pages.
2	CSS3	Designs and styles the interface.
3	JavaScript (ES6)	Handles map interaction, data loading and calculations in the browser.
4	Leaflet	Displays the interactive map, markers, circles and routes.
5	Python	Runs the backend and the optimisation algorithms.
6	FastAPI and Uvicorn	Provides the REST API between the frontend and the backend.
7	SQLite	Stores user accounts; passwords are saved as salted hashes.
8	scikit-learn	Trains the Random Forest risk classifier.
9	NetworkX	Builds the road graph used by the A* search.
10	GDACS, USGS, NASA EONET	Provide live disaster events.
11	OpenStreetMap and Overpass API	Provide hospitals, schools, halls and stadiums.
12	OSRM	Calculates driving routes and travel times.

3.3 ALGORITHMS USED

The project uses algorithms from different units of the Foundations of Artificial Intelligence syllabus. Table 3.2 lists each algorithm, where it is used, and the related unit.

Table 3.2 Algorithms and their purpose

Algorithm	Purpose in the project	Syllabus unit
Haversine distance	Shortlists the nearest candidate hospitals and shelters by straight-line distance.	Unit I
Shortest path on road network	Finds the fastest driving route and travel time (OSRM, Dijkstra family).	Units I and II
A* search	Plans routes on a road graph in the backend using a straight-line heuristic.	Unit II
Genetic algorithm	Allocates warehouse stock to zones, highest risk first, within stock limits.	Unit II
Hungarian algorithm	Assigns rescue teams to incidents with minimum total distance.	Unit II
Rule-based classification	Classifies events and zones as High, Medium or Low risk.	Unit IV
Random Forest	Predicts the risk level of an area from rainfall, water level, elevation and density.	Unit V

The risk of a zone is calculated as 0.6 × (severity ÷ 10) + 0.4 × (population ÷ largest population). A score of 0.6 or more is High, a score of 0.4 or more is Medium, and a lower score is Low. For events read from the live feeds, the GDACS alert colour is used directly (Red is High, Orange is Medium and Green is Low), and earthquakes from USGS are rated by magnitude.

CHAPTER 4
MODULES DESCRIPTION

4.1 User Authentication Module

The User Authentication Module controls access to the application. A new user registers with a name, email address, password and role, such as Citizen, Volunteer or Relief Officer. The backend checks that the email is valid and that the password has at least six characters. Passwords are never stored in plain form; each one is combined with a random salt and saved as a PBKDF2 hash in an SQLite database. On sign-in, the server returns a random session token, and this token is required for protected functions such as the allocation planner. This module keeps the relief data limited to registered users.

4.2 Live Disaster Data Module

The Live Disaster Data Module reads current events from three open services. GDACS gives alerts for cyclones, floods, earthquakes, volcanoes and forest fires. USGS gives earthquakes of magnitude 4.5 and above in the past week. NASA EONET gives open natural events such as wildfires, severe storms and floods. The module requests all three at the same time, and if one service is unavailable the others still load. Each event is converted to one common format containing the type, name, latitude, longitude, date, source and risk level. These events are then drawn as markers on the map, and counters show how many events belong to each type.

4.3 Disaster Management Module

The Disaster Management Module is the first category in the search box. When it is chosen, the live events appear as coloured markers: red for High, orange for Medium and green for Low risk. A set of tracker chips for earthquakes, cyclones and storms, floods, wildfires, volcanoes, landslides and drought lets the user filter the map by disaster type. Clicking a marker opens an incident page. This page describes what is happening, the time in GMT and local time, the location and

the source, together with a link to the official report. It also shows the estimated number of people in the affected area, survivors, injured, displaced and casualties, and a table of prone zones. The zones are High, Medium and Low according to the distance from the centre, with the recommended action for each.

4.4 Hospital Module

The Hospital Module helps to send a patient to the right hospital quickly. When Hospitals is chosen, the hospitals of the visible area are read from OpenStreetMap and shown as markers. The user then clicks on the map to mark an emergency. The system selects the 24 hospitals that are nearest in a straight line and asks the routing service for the real driving time from each hospital to the emergency point in a single request. The three hospitals with the shortest road time are ranked, their routes are drawn, and the fastest one is shown as a thick blue line with its time and distance, while the other routes are shown in grey. A summary box lists the three hospitals and the estimated ambulance time with the siren on. Clicking a hospital opens a detail page that shows its facilities, ambulance and bed capacity, the turn-by-turn route and a button to open the same route in Google Maps.

4.5 Shelter Module

The Shelter Module finds safe places for affected families. The user clicks on the affected area, and the system draws a danger zone around that point. It then searches OpenStreetMap for named schools, colleges, community halls, function halls, places of worship and stadiums outside the danger zone, because these buildings are commonly used as relief shelters. The candidates are ranked by road travel time, and only the best five are displayed so that the map is not crowded. The module accepts the number of people who need shelter and fills the shelters one by one up to their capacity. If the capacity is not enough, it reports the shortfall and the number of relief tents needed. The shelter detail page shows

capacity, current occupancy, facilities, the supplies needed and the evacuation route.

4.6 Warehouse Module

The Warehouse Module shows the stores from which relief material can be sent. When it is chosen, the map shows the warehouses for every state, and a message gives the number of warehouses that are displayed. Each warehouse has a stock of food and water. For an affected area, the nearest warehouse is found and the share of the area's need that it can meet is calculated. When several areas compete for limited stock, a genetic algorithm decides how much each warehouse sends to each zone. The fitness of a solution rewards supplying zones with a high risk score, penalises long distances and gives a very large penalty to any plan that sends more than a warehouse holds. A repair step then removes any remaining excess and uses leftover stock for the highest-risk zones.

4.7 Rescue Team Dispatch Module

The Rescue Team Dispatch Module deals with several incidents at the same time. Each incident has a type, such as flood, fire, earthquake or cyclone, and a severity. The module builds a table of the distance between every rescue team and every incident and uses the Hungarian algorithm to find the assignment with the minimum total distance. The skill of each team is compared with the type of the incident. The time complexity of the algorithm is $O(n^3)$, which is small for the number of teams used. Every assignment becomes a ticket that shows the priority, distance, estimated time of arrival and skill match, and the operator can dispatch, reassign or resolve it.

4.8 Ticketing and Resource Planning Module

The Ticketing and Resource Planning Module turns the information about a disaster into actions that can be followed. For an affected area, the number of people and the number of days are used to estimate food, drinking water and other

supplies such as medical kits, blankets, tents, hygiene kits, torches and rescue boats. The list depends on the type of disaster and on the risk level: High zones receive essential, shelter and rescue items, Medium zones receive essential and shelter items, and Low zones receive essential items only. Each damage and resource requirement is stored as a ticket with a priority, a team and a status of Open, In progress or Resolved. New tickets can be added, and the status is saved so that the progress of relief work can be reviewed.

4.9 User Interface Module

The User Interface Module provides the full-screen map interface. A search box at the top left opens a list of categories, and the chosen category is shown on the map. The pointer changes to a special icon when the user is marking an emergency, and information boxes show the result of each action. Detail pages for incidents, hospitals and shelters open from the markers and have a button to return to the map. The interface uses only HTML, CSS and JavaScript with Leaflet, and it works in a standard web browser without installing any software on the user's computer.

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The backend was developed in Python, and its libraries are listed in the requirements file. They are installed with the command pip install -r requirements.txt. The backend server is then started with python -m uvicorn main:app --reload, and it listens on port 8000. The frontend files are opened with the Live Server extension of Visual Studio Code on port 5500, or they can be served with python -m http.server. The browser needs an internet connection to read the live disaster feeds, the map tiles, the hospital data and the road routes.

The live feeds, OpenStreetMap data and OSRM routes are free public services. Their response time depends on the internet connection and on the load of the service. If one service does not respond, the system shows a message and the user can try again after a few seconds. Table 5.1 summarises the setup.

Table 5.1 Experimental setup

S.No.	Component	Description
1	Programming languages	Python 3 and JavaScript (ES6)
2	Frontend technologies	HTML5, CSS3, JavaScript and Leaflet 1.9.4
3	Backend framework	FastAPI with Uvicorn server (port 8000)
4	Database	SQLite for user accounts
5	Machine learning	scikit-learn Random Forest classifier
6	Optimisation and search	Genetic algorithm, A* search (NetworkX) and Hungarian algorithm
7	Live disaster data	GDACS, USGS and NASA EONET
8	Map and facility data	OpenStreetMap through the Overpass API; map tiles by Esri
9	Routing service	OSRM public routing service
10	Development environment	Visual Studio Code with Live Server (port 5500)

S.No.	Component	Description
11	Browser	Google Chrome
12	Input	Category selected in the search box and a click on the map
13	Output	Markers, blue routes, detail pages, supply lists and tickets

5.2 RESULTS

The application opens as a full-screen world map. A search box is placed at the top left with the text “Search Reliefline (click to choose a category)”. The zoom buttons and a layer button are at the bottom of the map. Choosing a category from the list displays the matching information on the map, as shown in Figure 5.1.

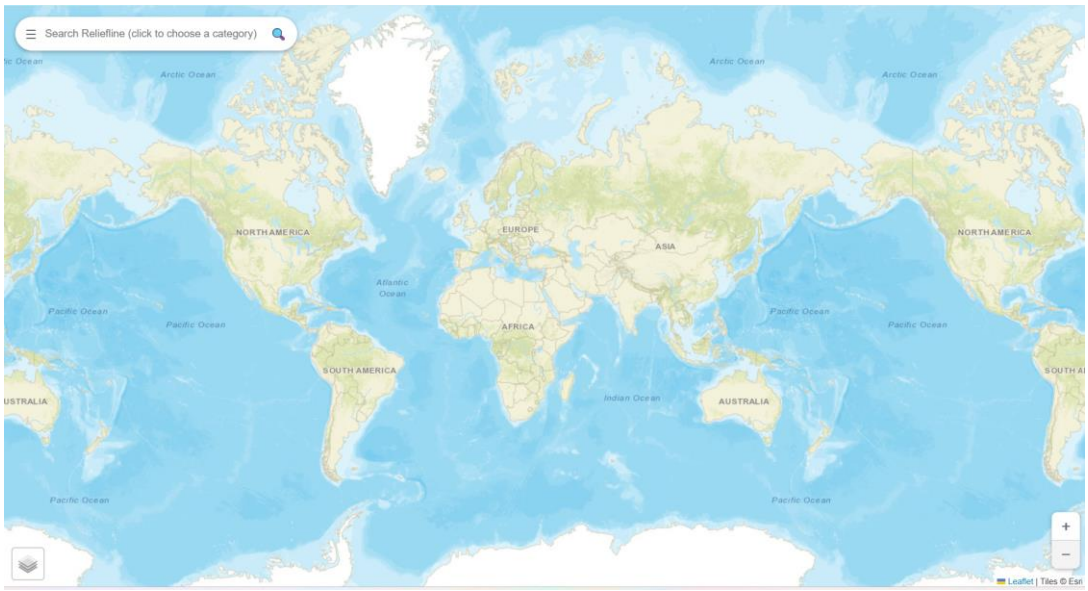


Figure 5.1 Main map interface with the category search box

In the Hospital Module, the hospitals of the selected part of Chennai are shown as markers. After an emergency was marked on the map, the system ranked the nearest hospitals by road travel time and drew the fastest route as a blue line with its label. The box at the bottom left names the hospitals in order, as given in Table 5.2. Figure 5.2 shows the result.

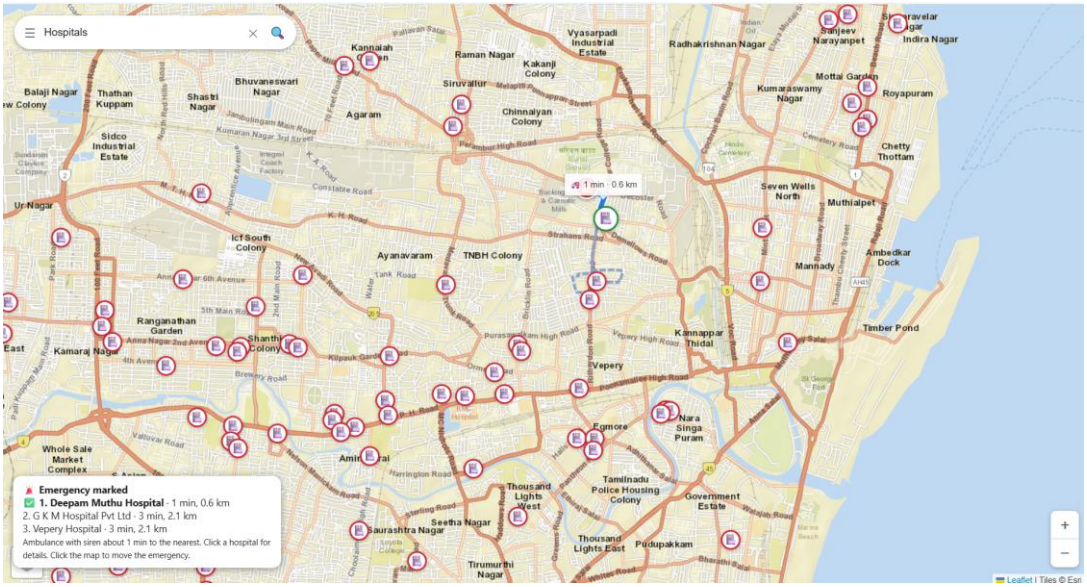


Figure 5.2 Hospital Module: emergency marked and fastest route to the nearest hospital

Table 5.2 Hospitals ranked for the marked emergency

Rank	Hospital	Travel time	Distance
1	Deepam Muthu Hospital	1 min	0.6 km
2	G K M Hospital Pvt Ltd	3 min	2.1 km
3	Vepery Hospital	3 min	2.1 km

The nearest hospital in a straight line is not always the fastest by road, so the ranking uses the real road time. In this run, the first hospital was reached in about one minute over 0.6 km, and the estimated ambulance time with the siren on was also about one minute. The second and third hospitals were three minutes away. Clicking the first hospital opened its detail page, shown in Figure 5.3.

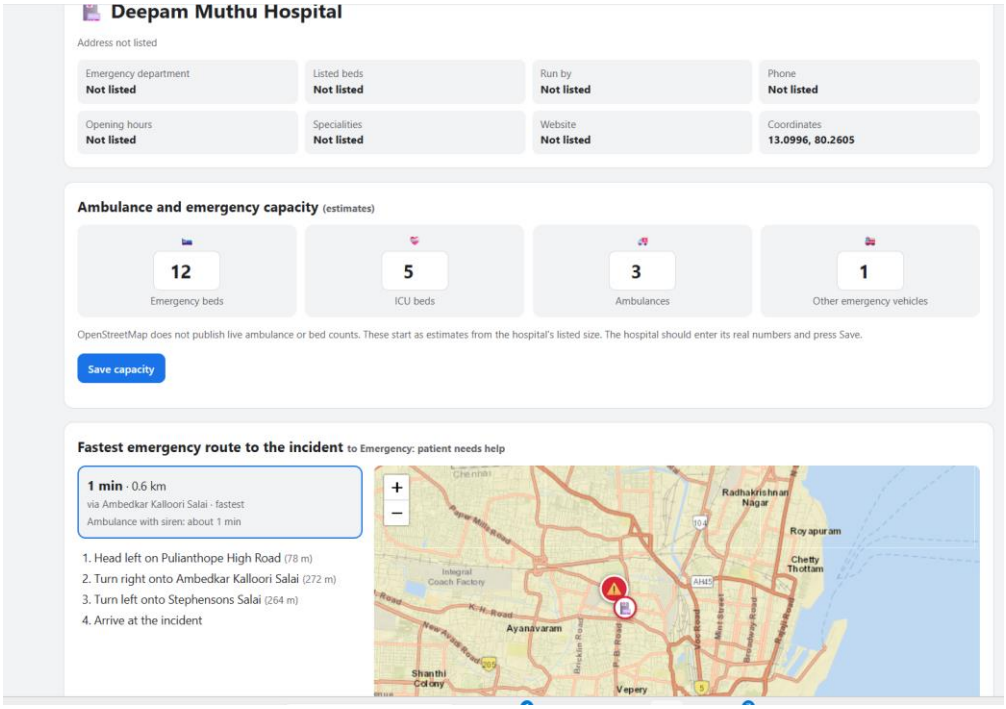


Figure 5.3 Hospital detail page with capacity and route steps

The detail page lists the information that is available for the hospital. Items that OpenStreetMap does not contain, such as address, phone number, opening hours and website, are marked as “Not listed”. The ambulance and emergency capacity shows 12 emergency beds, 5 ICU beds, 3 ambulances and 1 other emergency vehicle. These values are estimates based on the hospital’s size, and the hospital can enter its real numbers and save them. The route section shows the fastest route of 1 minute and 0.6 km via Ambedkar Kalloori Salai, with the steps: head left on Pulianthope High Road (78 m), turn right onto Ambedkar Kalloori Salai (272 m), turn left onto Stephensons Salai (264 m), and arrive at the incident. A button opens the same route in Google Maps.

In the Warehouse Module, the map was zoomed to India, and warehouse markers were displayed for the states. The message at the bottom of the screen reported that 28 warehouses were shown, one per state, as in Figure 5.4. From these markers, the nearest warehouse to an affected area can be selected and its stock compared with the supplies required.

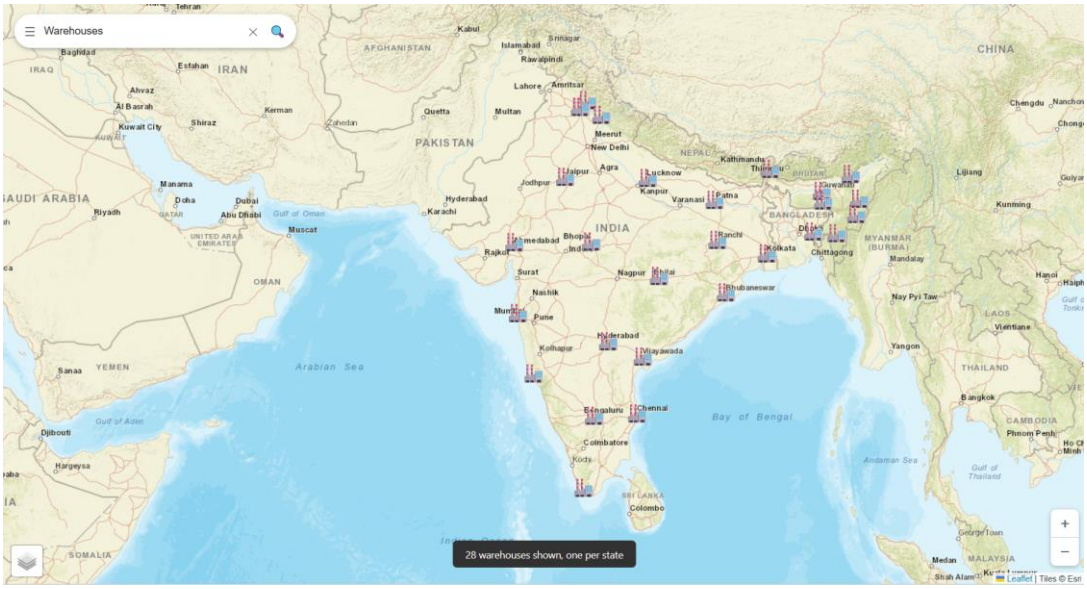


Figure 5.4 Warehouse Module: 28 warehouses shown, one per state

The following observations were made from the results:

- The category search, markers and detail pages worked together from a single map screen.
- The fastest hospital was selected by real road time, and its route was drawn on the map in blue.
- The unavailable details of a hospital were shown clearly as “Not listed” and not guessed.
- The warehouse markers gave a state-wise view of the relief stores across India.

The results depend on the data of OpenStreetMap, which is created by volunteers and may be incomplete. The travel times do not include live traffic, and the ambulance, bed and population figures are estimates until they are replaced by real values. For this reason the system should be used as a decision-support tool and not as an official record.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The AI Disaster Relief Resource Optimizer was developed to help relief officers take faster and better-organised decisions during a disaster. The system combines live disaster information, hospitals, shelters, warehouses, rescue teams and ticket tracking in one web application that is based on an interactive world map. By reading real events from GDACS, USGS and NASA EONET, the system always works with current information, and each event is classified as High, Medium or Low risk.

The emergency feature allows the user to click on the map and see the nearest hospital by real road travel time, with the fastest route drawn in blue and the other options shown for comparison. The hospital detail page gives the available facilities, the capacity, and the turn-by-turn route. The shelter and warehouse modules extend the same approach to evacuation and supply planning, and the estimated needs are converted into tickets that can be followed until they are resolved.

The project applies several topics of the Foundations of Artificial Intelligence course: search algorithms for routing, a genetic algorithm for the allocation of limited stock, the Hungarian algorithm for assigning rescue teams, rule-based classification of risk, and a Random Forest model for risk prediction. Because it is built on open data and free services, the system can be used without any paid software. Overall, the project shows how artificial intelligence and open data can reduce the time needed to decide where help should be sent first.

6.2 FUTURE WORK

Several improvements are possible. The first is to use live traffic and road-closure information so that the travel times and routes match the real conditions during a disaster. The second is to connect the hospital and shelter modules to real data,

such as the number of free beds, ambulances and occupied shelter places, which hospitals and shelter managers would update directly.

The accuracy of the risk and damage estimates can be improved by training the Random Forest model on real historical disaster data and by using official population data for each area. The A* search can be run on a complete road graph of the affected area, and the warehouse allocation can be extended to include delivery vehicles and delivery time. Other possible enhancements include sending alert messages to residents by SMS, support for regional languages, a mobile application, and the ability to work with saved data when the internet is not available.

In conclusion, the AI Disaster Relief Resource Optimizer provides a foundation for an intelligent decision-support tool for disaster response. With real capacity data, live traffic and wider testing, it has the potential to become a practical system for emergency control rooms.

CHAPTER 7
REFERENCES

1. Agafonkin, V. (n.d.) *Leaflet: an open-source JavaScript library for interactive maps*, available at: https://leafletjs.com/.

2. Altay, N. and Green, W.G. (2006) 'OR/MS research in disaster operations management', *European Journal of Operational Research*, Vol.175, No.1, pp.475–493.

3. Breiman, L. (2001) 'Random forests', *Machine Learning*, Vol.45, No.1, pp.5–32.

4. Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', *Numerische Mathematik*, Vol.1, No.1, pp.269–271.

5. Galindo, G. and Batta, R. (2013) 'Review of recent developments in OR/MS research in disaster operations management', *European Journal of Operational Research*, Vol.230, No.2, pp.201–211.

6. GDACS (n.d.) *Global Disaster Alert and Coordination System*, available at: https://www.gdacs.org/.

7. Haklay, M. and Weber, P. (2008) 'OpenStreetMap: user-generated street maps', *IEEE Pervasive Computing*, Vol.7, No.4, pp.12–18.

8. Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics*, Vol.4, No.2, pp.100–107.

9. Holland, J.H. (1975) *Adaptation in Natural and Artificial Systems*, University of Michigan Press, Ann Arbor.

10. Kuhn, H.W. (1955) 'The Hungarian method for the assignment problem', *Naval Research Logistics Quarterly*, Vol.2, No.1–2, pp.83–97.

11. Luxen, D. and Vetter, C. (2011) 'Real-time routing with OpenStreetMap data', *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pp.513–516.

12. Munkres, J. (1957) 'Algorithms for the assignment and transportation problems', *Journal of the Society for Industrial and Applied Mathematics*, Vol.5, No.1, pp.32–38.

13. NASA (n.d.) *Earth Observatory Natural Event Tracker (EONET)*, available at: https://eonet.gsfc.nasa.gov/.

14. Overpass API (n.d.) *Overpass API documentation*, OpenStreetMap Wiki, available at: https://wiki.openstreetmap.org/wiki/Overpass_API.

15. Özdamar, L., Ekinci, E. and Küçükyazici, B. (2004) 'Emergency logistics planning in natural disasters', *Annals of Operations Research*, Vol.129, No.1–4, pp.217–245.